Technical Documentation

Distributed Systems and Cloud Computing

Meenakshi Sridharan Sundaram

Contents

1 Virtual Machines and Containers **4**
1.1 Objective . . . 4
1.2 Virtual Machine Setup . . . 4
1.2.1 Installation Requirements . . . 4
1.2.2 VM Configuration . . . 4
1.3 Instance Provisioning . . . 4
1.3.1 Pets vs. Cattle Methodology . . . 4
1.3.2 Provisioning Steps . . . 4
1.4 Docker Fundamentals . . . 4
1.5 Implementation Tasks . . . 5
1.5.1 Task 1: Modifying hello.go . . . 5
1.5.2 Task 2: Docker Image Execution . . . 5
1.6 Key Concepts . . . 5
1.6.1 Question 1: Pets vs. Cattle . . . 5
1.6.2 Question 2: sudo Requirement . . . 5
1.6.3 Question 3: Docker Image Sources . . . 6
1.6.4 Question 4: Docker Images vs. Containers . . . 6

2 Service Containerization **7**
2.1 Objective . . . 7
2.2 Key-Value Store Orchestration . . . 7
2.2.1 Project Setup . . . 7
2.2.2 Architecture . . . 7
2.2.3 Docker Compose Management . . . 7
2.3 Network and Communication . . . 7
2.4 Testing the Key-Value Store . . . 7
2.5 Technical Questions . . . 8
2.5.1 Question 1: Original Code License . . . 8
2.5.2 Question 2: GNU GPL vs. GNU AGPL . . . 8
2.5.3 Question 3: Connection Retry Logic . . . 8
2.6 Environment Variable Management . . . 8
2.6.1 Motivation . . . 8
2.6.2 Modified service.go . . . 9
2.6.3 Updated docker-compose.yml . . . 9
2.7 Fault Tolerance Testing . . . 10
2.7.1 Test 1: KVS Service Failure . . . 10
2.7.2 Test 2: PostgreSQL Service Failure . . . 10
2.8 Data Loss Prevention Strategies . . . 10
2.8.1 Strategy 1: Retry Logic . . . 10
2.8.2 Strategy 2: Graceful Degradation . . . 10
2.8.3 Strategy 3: Database Replication . . . 11

3 Cassandra Cluster on Docker **12**
3.1 Introduction . . . 12
3.2 Key Components . . . 12
3.2.1 Docker . . . 12
3.2.2 Docker Compose . . . 12
3.2.3 Cassandra Cluster . . . 12
3.3 Cluster Setup . . . 12
3.3.1 Docker Compose Configuration . . . 12

3.3.2 Building and Starting . . . 13
3.4 Cassandra Management . . . 13
3.4.1 Nodetool . . . 13
3.4.2 CQLSH . . . 13
3.5 Data Storage . . . 13
3.6 Tunable Consistency . . . 13
3.6.1 Consistency Levels . . . 13
3.7 Configuration Questions . . . 14
3.7.1 Question 1: CASSANDRA_CLUSTER_NAME Misconfiguration . . . 14
3.7.2 Question 2: CASSANDRA_SEEDS Purpose . . . 14
3.7.3 Question 3: UN Status Meaning . . . 14
3.7.4 Question 4: Data Storage Location . . . 14
3.8 Consistency Experiments . . . 14
3.8.1 Experiment 1: ONE/ONE Consistency . . . 14
3.8.2 Experiment 2: QUORUM/QUORUM Consistency . . . 14
3.8.3 Experiment 3: ALL/ONE Consistency . . . 14
3.8.4 Question: QUORUM Write, ONE Read . . . 15
3.8.5 Question: Running from VM Directly . . . 15
3.9 Handling Failures . . . 15
3.9.1 Failure Simulation . . . 15

4 Container Orchestration with Kubernetes 16
4.1 Introduction . . . 16
4.2 Key Concepts . . . 16
4.2.1 Kubernetes . . . 16
4.2.2 kind . . . 16
4.2.3 StatefulSets . . . 16
4.2.4 Deployments . . . 16
4.2.5 Services . . . 16
4.3 Cluster Setup with kind . . . 16
4.3.1 Configuration . . . 16
4.4 Cassandra Service Deployment . . . 17
4.4.1 StatefulSet Configuration . . . 17
4.4.2 Deployment Commands . . . 17
4.5 Technical Questions . . . 18
4.5.1 Question 1: Resources Section . . . 18
4.5.2 Question 2: Environment Variables . . . 18
4.5.3 Question 3: Service Discovery . . . 18
4.6 Writer Application Deployment . . . 18
4.6.1 Deployment Configuration . . . 18
4.6.2 Build and Deploy . . . 18
4.7 Application Questions . . . 19
4.7.1 Question 1: Connection to Cassandra . . . 19
4.7.2 Question 2: Troubleshooting CrashLoopBackOff . . . 19
4.8 Stateless Application Enhancement . . . 19
4.8.1 Modified writer.go . . . 19
4.8.2 Consistency Considerations . . . 19

5 Distributed Message Queues with Kafka 20
5.1 Introduction . . . 20
5.2 Kafka Services . . . 20
5.2.1 Architecture Components . . . 20

5.2.2 Cluster Setup . . . 20
5.2.3 Broker Verification . . . 20
5.3 Technical Questions . . . 20
5.3.1 Question 1: ZooKeeper Location . . . 20
5.3.2 Question 2: Broker Discovery . . . 20
5.3.3 Question 3: localhost Reference . . . 20
5.4 Topics and Messages . . . 21
5.4.1 Kafka Concepts . . . 21
5.4.2 Producer/Consumer Deployment . . . 21
5.4.3 Topic Inspection . . . 21
5.5 Client Configuration . . . 21
5.5.1 clients.yml Structure . . . 21
5.5.2 Configuration Questions . . . 22
5.6 Enhanced Consumer Implementation . . . 22
5.6.1 Approach . . . 22
5.6.2 Implementation . . . 22
5.6.3 Validation . . . 23
5.7 Discussion . . . 23
5.7.1 ConsumerGroup Advantages . . . 23
5.7.2 Scalability Considerations . . . 24
5.8 Summary . . . 24

1 Virtual Machines and Containers

1.1 Objective

This project establishes environments for software development and deployment using virtual machines (VMs) and containers, focusing on VirtualBox for VM management, SSH access, instance provisioning, and Docker for container management.

1.2 Virtual Machine Setup

1.2.1 Installation Requirements

- Verify VirtualBox compatibility (Apple Silicon platforms not supported)
- Install latest VirtualBox version with hardware virtualization enabled

1.2.2 VM Configuration

1. Download and verify the .ova appliance file
2. Import VM appliance into VirtualBox
3. Configure network settings: Guest Port "number" → Host Port "number"
4. Start VM and access via SSH:

```
    ssh ubuntu@"enter your port number"
    # Password: "enter your password"

```

1.3 Instance Provisioning

1.3.1 Pets vs. Cattle Methodology

Pets: Servers treated as unique entities requiring individual care and maintenance (traditional approach).

Cattle: Servers managed as a group, easily replaceable using automation scripts (modern cloud computing approach).

1.3.2 Provisioning Steps

```
# Clone repository
git clone https://github.com/wngjia/ece573-prj01.git

# Provision instance
sudo ece573-prj01/setup_vm.sh

# Verify installations
go version
docker version
```

1.4 Docker Fundamentals

Docker provides lightweight containers as an alternative to VMs, running isolated applications with minimal resource usage.

```
# Obtain Go development image
docker pull golang:1.21

# Build custom Docker images using Dockerfile
```

1.5 Implementation Tasks

1.5.1 Task 1: Modifying hello.go

Modified `hello.go` to print current date/time and last four CWID digits:

```go
package main

import (
    "fmt"
    "time"
)

func main() {
    cwid := "A20592581"
    lastFour := cwid[len(cwid)-4:]

    currentTime := time.Now()

    fmt.Printf("Hello World!\nCurrent date and time: %s\n",
               currentTime.Format("2006-01-02 15:04:05"))
    fmt.Printf("Last four digits of CWID: %s\n", lastFour)
}
```

1.5.2 Task 2: Docker Image Execution

Build and run the updated Docker image:

```
# Build Docker image
docker build -t prj01 .

# Run container
docker run prj01
```

1.6 Key Concepts

1.6.1 Question 1: Pets vs. Cattle

- **Pets:** Unique, carefully managed, irreplaceable servers requiring individual attention
- **Cattle:** Generic, replaceable, disposable servers supporting scalable, resilient infrastructure

1.6.2 Question 2: sudo Requirement

The `setup_vm.sh` script requires `sudo` because it performs administrative tasks:

- Installing software packages
- Modifying system configurations
- Accessing restricted files and directories

1.6.3 Question 3: Docker Image Sources

Docker images are pulled from:

- **Docker Hub:** Default public registry
- **Alternative registries:** Amazon ECR, Google Container Registry, private registries

1.6.4 Question 4: Docker Images vs. Containers

- **Docker Image:** Lightweight, standalone executable package containing code, runtime, libraries, and configurations (snapshot of container filesystem)
- **Docker Container:** Runtime instance of a Docker image, running the application in an isolated environment

2 Service Containerization

2.1 Objective

Learn service containerization using Docker tools, focusing on Docker Compose for managing and connecting multiple services through container orchestration with a key-value store backed by PostgreSQL.

2.2 Key-Value Store Orchestration

2.2.1 Project Setup

```
# Clone repository
git clone https://github.com/wngjia/ece573-prj02.git

# Execute setup script
./setup_vm.sh
```

2.2.2 Architecture

The key-value store consists of two microservices:

- **kvs service:** Uses source code from textbook
- **postgres service:** Uses pre-built PostgreSQL image

2.2.3 Docker Compose Management

```
# Build container images
docker compose build

# Create containers and private network
docker compose create

# Start containers in background
docker compose start

# Verify status
docker ps -a
```

2.3 Network and Communication

Docker Compose automatically creates a private network for containers. Each service container is accessible using its service name as the domain name through Docker's built-in DNS service.

2.4 Testing the Key-Value Store

```
# Retrieve value
curl -X GET 127.0.0.1:8080/v1/a

# Set value
curl -X PUT -d "value a" 127.0.0.1:8080/v1/a

# Check logs
docker logs kvs
```

2.5 Technical Questions

2.5.1 Question 1: Original Code License

The original example code uses **Apache License 2.0**, a permissive open-source license allowing both open-source and proprietary use while providing protections to users and developers.

2.5.2 Question 2: GNU GPL vs. GNU AGPL

GNU GPL (General Public License):

- Ensures freedom to run, study, modify, and share software
- Distribution of modified software requires distributing source code under GPL terms
- **SaaS Loophole:** Running GPL software as a web service doesn't require source code distribution

GNU AGPL (Affero General Public License):

- Closes the SaaS loophole
- Network interaction with software requires making source code available
- Applies when providing cloud services, not just direct distribution

Key Difference for Cloud Services:

- GPL: No obligation to share source code for cloud services
- AGPL: Must share source code when providing network-accessible services

2.5.3 Question 3: Connection Retry Logic

The kvs service needs retry logic because Docker Compose ensures service *start order* but not service *readiness*:

- PostgreSQL may take time to initialize after starting
- kvs attempts connection before PostgreSQL is ready to accept connections
- Retry mechanism ensures eventual connection once PostgreSQL is operational

2.6 Environment Variable Management

2.6.1 Motivation

Replacing hard-coded database parameters with environment variables provides:

- Flexibility for different environments (development, testing, production)
- Enhanced security by not exposing sensitive information in code
- Easy configuration management

2.6.2 Modified service.go

```go
import (
    "os"
    "fmt"
    "log"
)

func connectDB() {
    host := os.Getenv("POSTGRES_HOST")
    if host == "" {
        host = "postgres" // default value
    }

    dbName := os.Getenv("POSTGRES_DB")
    if dbName == "" {
        dbName = "kvs"
    }

    user := os.Getenv("POSTGRES_USER")
    if user == "" {
        user = "test"
    }

    password := os.Getenv("POSTGRES_PASSWORD")
    if password == "" {
        password = "hunter2"
    }

    connStr := fmt.Sprintf("host=%s dbname=%s user=%s password=%s",
                           host, dbName, user, password)

    log.Printf("Connecting to database at %s", host)
    // Connection logic...
}
```

2.6.3 Updated docker-compose.yml

```yaml
version: '3'
services:
  postgres:
    image: postgres:11
    environment:
      - POSTGRES_USER=test
      - POSTGRES_PASSWORD=hunter2
      - POSTGRES_DB=kvs

  kvs:
    build: kvs
    environment:
      - POSTGRES_HOST=postgres
      - POSTGRES_DB=kvs
      - POSTGRES_USER=mena
      - POSTGRES_PASSWORD=abcdefghi
    ports:
      - "8080:8080"
    depends_on:
      - postgres
```

2.7 Fault Tolerance Testing

2.7.1 Test 1: KVS Service Failure

```
# Store key-value pair
curl -X PUT -d "value=a" 127.0.0.1:8080/v1/a

# Shutdown kvs service
docker kill ece573-prj02-kvs-1

# Restart kvs service
docker compose start

# Verify data persistence
curl -X GET 127.0.0.1:8080/v1/a
```

Result: No data loss. Data persists in PostgreSQL database, unaffected by kvs service shutdown.

2.7.2 Test 2: PostgreSQL Service Failure

```
# Store first key-value pair
curl -X PUT -d "value=a" 127.0.0.1:8080/v1/a

# Shutdown PostgreSQL
docker kill ece573-prj02-postgres-1

# Attempt to store second pair (will fail)
curl -X PUT -d "value=b" 127.0.0.1:8080/v1/b

# Restart PostgreSQL
docker compose start

# Verify first pair exists
curl -X GET 127.0.0.1:8080/v1/a

# Verify second pair lost
curl -X GET 127.0.0.1:8080/v1/b
```

Result: Partial data loss. First pair persists, second pair lost because PostgreSQL was unavailable during write operation.

2.8 Data Loss Prevention Strategies

2.8.1 Strategy 1: Retry Logic

Implement retry mechanisms in kvs service to queue write operations when PostgreSQL is unavailable and retry when database is back online.

2.8.2 Strategy 2: Graceful Degradation

Modify kvs service to handle PostgreSQL downtime gracefully by:

- Returning meaningful error messages to clients
- Queuing requests temporarily

2.8.3 Strategy 3: Database Replication

Set up PostgreSQL replication or clustering for high availability:

- If one instance goes down, another takes over
- Prevents data loss and minimizes downtime

3 Cassandra Cluster on Docker

3.1 Introduction

Apache Cassandra in a Docker environment simplifies distributed database deployment by using containers instead of physical/virtual servers. Docker provides consistent environments for development and testing, enabling distributed system simulation on a single machine.

3.2 Key Components

3.2.1 Docker

Platform for packaging applications and dependencies into containers—lightweight, standalone units running consistently across environments.

3.2.2 Docker Compose

Tool for defining and running multi-container applications using YAML configuration files to manage services, networking, and client applications.

3.2.3 Cassandra Cluster

NoSQL distributed database designed for handling large data volumes across many servers. Each container acts as an independent Cassandra node.

3.3 Cluster Setup

3.3.1 Docker Compose Configuration

```yaml
version: '3'
services:
  db1:
    image: cassandra:latest
    environment:
      - CASSANDRA_CLUSTER_NAME=TestCluster
      - CASSANDRA_SEEDS=db1,db2,db3
    networks:
      - cassandra_net

  db2:
    image: cassandra:latest
    environment:
      - CASSANDRA_CLUSTER_NAME=TestCluster
      - CASSANDRA_SEEDS=db1,db2,db3
    networks:
      - cassandra_net

  db3:
    image: cassandra:latest
    environment:
      - CASSANDRA_CLUSTER_NAME=TestCluster
      - CASSANDRA_SEEDS=db1,db2,db3
    networks:
      - cassandra_net

  client:
    build: ./client
    depends_on:
      - db1
```

```
      - db2
      - db3
    networks:
      - cassandra_net

networks:
  cassandra_net:
    driver: bridge
```

3.3.2 Building and Starting

```
# Build images
docker compose build

# Create containers
docker compose create

# Start cluster
docker compose start

# Verify status
docker compose ps
docker compose exec db1 nodetool status
```

3.4 Cassandra Management

3.4.1 Nodetool

Command-line tool providing statistics and operations for Cassandra nodes:

```
docker compose exec db1 nodetool status
```

Output shows node health and load distribution. `UN` status indicates **Up and Normal**.

3.4.2 CQLSH

Cassandra Query Language Shell for database interaction:

```
docker compose exec db1 cqlsh
```

3.5 Data Storage

Cassandra stores data in `/var/lib/cassandra/data` inside containers. Without volume mapping, data is lost when containers are removed or restarted.

```
# Inspect data location
docker compose exec db1 ls /var/lib/cassandra/data
```

3.6 Tunable Consistency

Cassandra offers tunable consistency, trading off between consistency, availability, and partition tolerance (CAP theorem).

3.6.1 Consistency Levels

- **ONE:** Single replica responds—prioritizes availability, risks inconsistency
- **QUORUM:** Majority of replicas confirm—balances consistency and availability
- **ALL:** All replicas confirm—ensures full consistency, reduces availability

3.7 Configuration Questions

3.7.1 Question 1: CASSANDRA_CLUSTER_NAME Misconfiguration

If `CASSANDRA_CLUSTER_NAME` values don't match across nodes, they won't recognize each other as part of the same cluster, resulting in separate independent instances instead of a unified cluster.

3.7.2 Question 2: CASSANDRA_SEEDS Purpose

`CASSANDRA_SEEDS` specifies initial contact points for node discovery and cluster formation. It's not necessary to list all servers—one or two seed nodes suffice for bootstrapping, as other nodes automatically discover the cluster.

3.7.3 Question 3: UN Status Meaning

In `nodetool status` output, **UN** means **Up and Normal**—the node is operational and participating in the cluster without issues.

3.7.4 Question 4: Data Storage Location

Cassandra stores data in container filesystem under `/var/lib/cassandra/data` by default. Without volume mapping, data is lost when containers stop or are removed.

3.8 Consistency Experiments

3.8.1 Experiment 1: ONE/ONE Consistency

```
# Writer with ONE consistency
docker compose exec client writer -consistency ONE

# Reader with ONE consistency
docker compose exec client reader -consistency ONE
```

Result: Reader reports missing data because writes aren't guaranteed to replicate to all nodes immediately. Reader may access a node without complete data.

3.8.2 Experiment 2: QUORUM/QUORUM Consistency

```
# Writer with QUORUM consistency
docker compose exec client writer -consistency QUORUM

# Reader with QUORUM consistency
docker compose exec client reader -consistency QUORUM
```

Result: No missing data. Majority of replicas confirm writes and reads, ensuring consistency.

3.8.3 Experiment 3: ALL/ONE Consistency

```
# Writer with ALL consistency
docker compose exec client writer -consistency ALL

# Reader with ONE consistency
docker compose exec client reader -consistency ONE
```

Result: Potential inconsistent data. Writer waits for all replicas, but reader accesses only one, which may not be fully synchronized.

3.8.4 Question: QUORUM Write, ONE Read

With QUORUM writes and ONE reads, the writer ensures majority acknowledgment (stronger consistency), but the reader may still see stale data if it accesses a replica not yet updated.

3.8.5 Question: Running from VM Directly

Running writer and reader directly from VM is possible but requires all dependencies (Go, Cassandra driver, network configuration). Client container is pre-configured with these dependencies, simplifying usage.

3.9 Handling Failures

3.9.1 Failure Simulation

```
# Simulate node failure
docker compose kill db2
```

Cassandra continues serving requests depending on configured consistency level. Implement retry operations when failures occur:

```
for {
    err := session.Query(`INSERT INTO ...`).Exec()
    if err != nil {
        log.Printf("Write failed: %v. Retrying in 10 seconds...", err)
        time.Sleep(10 * time.Second)
    } else {
        break
    }
}
```

4 Container Orchestration with Kubernetes

4.1 Introduction

Kubernetes (K8s) automates containerized application deployment, scaling, and management. This project explores K8s through local cluster setup with `kind` (Kubernetes in Docker), deploying Cassandra as a StatefulSet and interacting with it using a stateless writer application.

4.2 Key Concepts

4.2.1 Kubernetes

Orchestration system enabling automated deployment, scaling, and operation of application containers across host clusters.

4.2.2 kind

Tool for running local Kubernetes clusters in Docker containers, simulating production-like environments for development and testing.

4.2.3 StatefulSets

Kubernetes objects managing stateful applications by providing unique identities to each pod—essential for databases requiring stable storage and network identities.

4.2.4 Deployments

Kubernetes objects managing stateless applications, enabling updates, rollbacks, and easy scaling through pod templates and replica specifications.

4.2.5 Services

Kubernetes objects providing stable endpoints and enabling communication between application components.

4.3 Cluster Setup with kind

4.3.1 Configuration

Create cluster with six worker nodes and one control plane:

```
kind create cluster --config cluster.yml
kind get nodes
```

cluster.yml:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
  - role: worker
  - role: worker
  - role: worker
  - role: worker
```

4.4 Cassandra Service Deployment

4.4.1 StatefulSet Configuration

```yaml
apiVersion: v1
kind: Service
metadata:
  name: cassandra-service
spec:
  clusterIP: None
  selector:
    app: cassandra
  ports:
    - port: 9042
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: cassandra
spec:
  serviceName: cassandra-service
  replicas: 5
  selector:
    matchLabels:
      app: cassandra
  template:
    metadata:
      labels:
        app: cassandra
    spec:
      containers:
      - name: cassandra
        image: cassandra:latest
        ports:
        - containerPort: 9042
        env:
        - name: CASSANDRA_CLUSTER_NAME
          value: "K8sCluster"
        - name: CASSANDRA_SEEDS
          value: "cassandra-0.cassandra-service"
        resources:
          requests:
            memory: "1Gi"
            cpu: "500m"
          limits:
            memory: "2Gi"
            cpu: "1000m"
```

4.4.2 Deployment Commands

```bash
# Apply configuration
kubectl apply -f cassandra.yml

# Verify deployment
kubectl get services
kubectl get statefulsets
kubectl get pods
```

4.5 Technical Questions

4.5.1 Question 1: Resources Section

The `resources` section specifies CPU and memory allocation:

- **Limits:** Maximum resources a container can utilize
- **Requests:** Minimum guaranteed resources

4.5.2 Question 2: Environment Variables

The `env` section defines Cassandra-specific variables. In Docker Compose, environment variables were set in the compose file. In Kubernetes, `cassandra.yml` manages them.

4.5.3 Question 3: Service Discovery

The Cassandra service identifies Pods using labels defined in the `selector` field of `cassandra.yml`, matching the `app: cassandra` label.

4.6 Writer Application Deployment

4.6.1 Deployment Configuration

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ece573-prj04-writer
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ece573-prj04-writer
  template:
    metadata:
      labels:
        app: ece573-prj04-writer
    spec:
      containers:
      - name: writer
        image: ece573-prj04-writer:v1
        env:
        - name: CASSANDRA_HOST
          value: "cassandra-service.default.svc.cluster.local"
        - name: TOPIC
          value: "test"
```

4.6.2 Build and Deploy

```bash
# Build and load image
./build.sh

# Deploy application
kubectl apply -f writer.yml

# Verify deployment
kubectl get deployment
kubectl get pods

```

```
# Check logs
kubectl logs -l app=ece573-prj04-writer
```

4.7 Application Questions

4.7.1 Question 1: Connection to Cassandra

The writer deployment connects to Cassandra using DNS name:

`cassandra-service.default.svc.cluster.local`

Kubernetes resolves this within the cluster network.

4.7.2 Question 2: Troubleshooting CrashLoopBackOff

Initial `CrashLoopBackOff` error was diagnosed using:

```
kubectl logs -l app=ece573-prj04-writer
```

Issue resolved by modifying the topic in `writer.yml`.

4.8 Stateless Application Enhancement

Since Deployments are stateless, the writer application restarts from scratch if the pod terminates. Enhanced writer reads last sequence number from Cassandra on restart for continuity.

4.8.1 Modified writer.go

```go
func getLastSequence(session *gocql.Session, topic string) int {
    var lastSeq int
    query := fmt.Sprintf("SELECT seq FROM ece573.prj04_last_seq WHERE topic='%s'", topic)

    if err := session.Query(query).Scan(&lastSeq); err != nil {
        log.Printf("No previous sequence found, starting from 0")
        return 0
    }

    log.Printf("Retrieved last sequence: %d", lastSeq)
    return lastSeq
}

func main() {
    // ... connection setup ...

    // Get last sequence
    seq := getLastSequence(session, topic)

    // Continue writing from last sequence
    for {
        seq++
        // ... write operations ...
    }
}
```

4.8.2 Consistency Considerations

If the writer fails to retrieve the latest sequence due to consistency issues, it may resume from an outdated sequence, causing duplicate entries. Maintaining consistency in Cassandra is essential to avoid conflicts.

5 Distributed Message Queues with Kafka

5.1 Introduction

Distributed message queues enable asynchronous communication and data streaming in modern distributed systems. Apache Kafka provides high-throughput, fault-tolerant messaging capabilities where producers send messages to topics and consumers retrieve them from specific partitions.

5.2 Kafka Services

5.2.1 Architecture Components

- **ZooKeeper:** Manages broker metadata and coordination
- **Kafka Cluster:** Brokers collaborate to distribute messages and ensure fault tolerance
- **Kubernetes:** Manages Kafka and ZooKeeper deployment as Pods, StatefulSets, and Services

5.2.2 Cluster Setup

```
# Reset and create cluster
./reset_cluster.sh

# Deploy Kafka and ZooKeeper
kubectl apply -f kafka.yml

# Verify services
kubectl get services
kubectl get pods
```

5.2.3 Broker Verification

```
# List brokers via ZooKeeper
kubectl exec zookeeper-0 -- zookeeper-shell localhost:2181 ls /brokers/ids

# View broker details
kubectl exec zookeeper-0 -- zookeeper-shell localhost:2181 get /brokers/ids/0
```

5.3 Technical Questions

5.3.1 Question 1: ZooKeeper Location

Kafka brokers use the `zookeeper.connect` property defined in `kafka.yml` to locate the ZooKeeper service.

5.3.2 Question 2: Broker Discovery

Brokers register themselves with ZooKeeper. ZooKeeper maintains metadata about all brokers, enabling them to discover and communicate with each other.

5.3.3 Question 3: localhost Reference

- In `zookeeper-shell`: localhost refers to the ZooKeeper Pod
- In `kafka-topics`: localhost refers to the Kafka Pod

5.4 Topics and Messages

5.4.1 Kafka Concepts

- **Topics:** Logical channels for message transmission
- **Partitions:** Topics split into partitions for scalability; messages in each partition are ordered
- **Producers:** Send messages to topics
- **Consumers:** Subscribe to topics or specific partitions

5.4.2 Producer/Consumer Deployment

```
# Build and deploy
./build.sh
kubectl apply -f clients.yml

# Check producer logs
kubectl logs -l app=ece573-prj05-producer

# Check consumer logs
kubectl logs -l app=ece573-prj05-consumer
```

5.4.3 Topic Inspection

```
# List topics
kubectl exec kafka-0 -- kafka-topics --bootstrap-server localhost:9092 --list

# Describe topic
kubectl exec kafka-0 -- kafka-topics --bootstrap-server localhost:9092 --
    describe --topic test
```

5.5 Client Configuration

5.5.1 clients.yml Structure

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ece573-prj05-producer
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ece573-prj05-producer
  template:
    spec:
      containers:
      - name: producer
        image: ece573-prj05-clients:v1
        env:
        - name: ROLE
          value: "producer"
        - name: KAFKA_BROKER
          value: "kafka-0.kafka-service.default.svc.cluster.local:9092"
        - name: TOPIC
          value: "test"
---
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ece573-prj05-consumer
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ece573-prj05-consumer
  template:
    spec:
      containers:
      - name: consumer
        image: ece573-prj05-clients:v1
        env:
        - name: ROLE
          value: "consumer"
        - name: KAFKA_BROKER
          value: "kafka-1.kafka-service.default.svc.cluster.local:9092"
        - name: TOPIC
          value: "test"
```

5.5.2 Configuration Questions

Question 1: ROLE and KAFKA_BROKER

- **ROLE:** Specifies whether application acts as producer or consumer
- **KAFKA_BROKER:** Specifies Kafka broker's address

Question 2: Sarama Library `github.com/shopify/sarama` is a high-performance Kafka client library for Go, widely adopted due to robust features and active maintenance.

Question 3: Changing Defaults Modify `num.partitions` and `default.replication.factor` in `kafka.yml` to change default partitions and replicas.

5.6 Enhanced Consumer Implementation

5.6.1 Approach

Modified `clients.go` to consume messages from all partitions efficiently:

1. Fetch topic metadata to identify partition count
2. Create consumer partition for each partition
3. Use goroutine for each consumer partition for parallel consumption
4. Use `select` statement to manage messages from multiple partitions

5.6.2 Implementation

```go
func consumer(broker, topic string) {
    consumer, err := sarama.NewConsumer([]string{broker}, nil)
    if err != nil {
        log.Fatalf("Cannot create consumer at %s: %v", broker, err)
    }
    defer consumer.Close()

    // Get partitions for the topic
    partitions, err := consumer.Partitions(topic)
```

```go
    if err != nil {
        log.Fatalf("Cannot fetch partitions for topic %s: %v", topic, err)
    }

    // Create channel for multiplexing messages
    messageChan := make(chan *sarama.ConsumerMessage)

    // Start goroutine for each partition
    for _, partition := range partitions {
        go func(partition int32) {
            partitionConsumer, err := consumer.ConsumePartition(
                topic, partition, sarama.OffsetNewest)
            if err != nil {
                log.Fatalf("Cannot create partition consumer for partition %d: %v",
                        partition, err)
            }
            defer partitionConsumer.Close()

            for msg := range partitionConsumer.Messages() {
                messageChan <- msg
            }
        }(partition)
    }

    log.Printf("%s: start receiving messages from all partitions at %s",
               topic, broker)

    // Process messages from channel
    for count := 1; ; count++ {
        msg := <-messageChan
        log.Printf("Received message from partition %d: %s",
                   msg.Partition, string(msg.Value))
        if count%1000 == 0 {
            log.Printf("%s: received %d messages", topic, count)
        }
    }
}
```

5.6.3 Validation

```bash
# Build updated consumer
./build.sh

# Deploy to cluster
kubectl apply -f clients.yml

# Verify consumer logs
kubectl logs -l app=ece573-prj05-consumer
```

Consumer logs confirm successful message consumption from all partitions without missing data.

5.7 Discussion

5.7.1 ConsumerGroup Advantages

Using `ConsumerGroup` interface provides:

- Dynamic partition assignment
- Automatic rebalancing when consumers are added/removed

- Fault tolerance in case of consumer failure
- Scalability for high-throughput scenarios

This approach eliminates hardcoded partition logic and supports multiple consumers sharing load across partitions.

5.7.2 Scalability Considerations

The implementation allows:

- Horizontal scaling by adding consumer instances
- Load distribution across partitions
- High availability through consumer group coordination

5.8 Summary

This project demonstrated deploying a Kafka cluster in Kubernetes, managing topics and partitions, and implementing a robust consumer for distributed message queue handling. All objectives were achieved with successful message processing across all partitions.